Artificial Intelligence

Lecture 07 – Markov Decision Process

Dr. Rajiv Misra, Professor
Dept. of Computer Science & Engg.
Indian Institute of Technology, Patna
rajivm@iitp.ac.in

Decisions - Decision Theory

- Decision Analysis (Present all possibilities)
- Decision Making (Choose the best one)

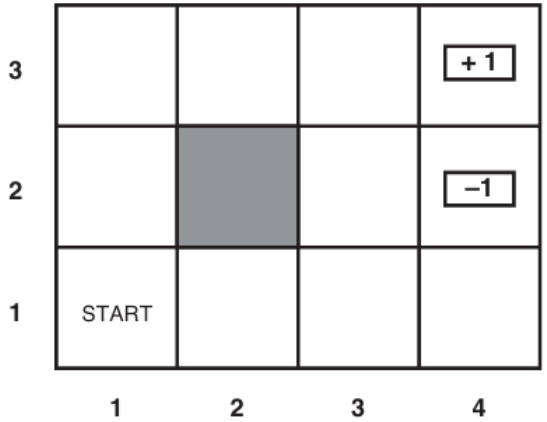
Utility Functions and Value of Information

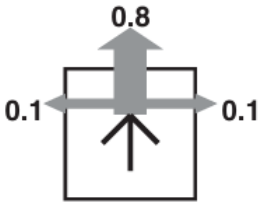
- Is paying more for a product a proxy for utility?
- if someone in the street stops you and offers:
  - 1. 100% chance of $300K or 80% chance of $400K.
  - 2. 25% chance of $300K or 20% chance of $400K
- Value of Information vs. Likelihood (test for Cancer w/small chance you may have it)

Scenario

Robot Game: A sequential decision problem

A simple 4×3 environment that presents the agent with a sequential decision problem.





- Transition model of the environment: the “intended” outcome occurs with probability 0.8, but with probability 0.2 the agent moves at right angles to the intended direction.
- A collision with a wall results in no movement.
- The two terminal states have reward +1 and–1, respectively, and all other states have a reward of–0.04.

Markov Decision Processes(MDP)

- The Robot (Agent) wants to leave the Grid.
- Each cell is a state it can be in
- At each state it can follow an action
- For each state and action there is a transition model P(s‘ l s, a)
- For each state and action there is a reward function R(s, a, s’) or R(s)

Markov Decision Processes(MDP)

- What does the solution look like?
- Specify what the agent should do in any state – the policy
- It wants to find the optimal policy π^* measured by the highest expected utility

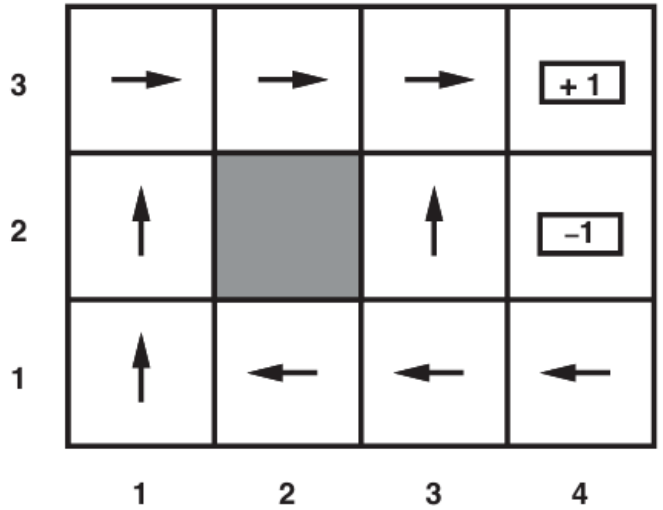
Optimal Policies

Given π^*

Agent determines that it is in state s

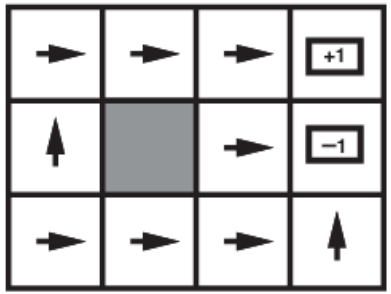
Agent takes the action given by $\pi^*(s)$

For example: if R(s) = -0.04 for all states

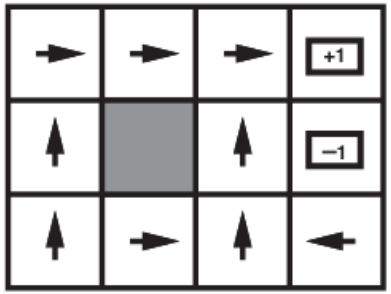


Optimal Policies

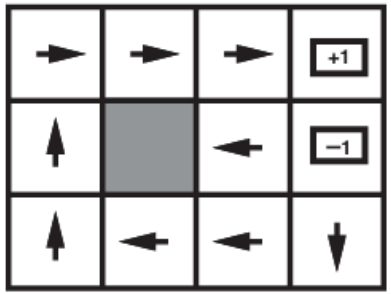
A balancing act b/w risk and reward



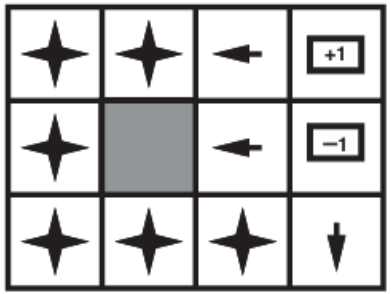
$R(s) < -1.6284$



$-0.4278 < R(s) < -0.0850$



$-0.0221 < R(s) < 0$



$R(s) > 0$

Utilities – over time

If you have a finite number of steps, your optimal policy changes.

What would be the optimal policy if you had 3 steps in total?

What if you had 2340 steps?

Utilities - Discounted Rewards

1. **Additive rewards**: The utility of a state sequence is

$$U_h([s_0, s_1, s_2, \ldots]) = R(s_0) + R(s_1) + R(s_2) + \cdots .$$

2. **Discounted rewards**: The utility of a state sequence is

$$U_h([s_0, s_1, s_2, \ldots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots ,$$

: **discount factor** γ is a number between 0 and 1

$$U_h([s_0, s_1, s_2, \ldots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max}$$

Policies and Utilities

The expected utility obtained by executing policy π starting in state s is

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t)\right]$$

And the optimal policy for state s is:

$$\pi_s^* = \underset{\pi}{\operatorname{argmax}}\, U^{\pi}(s)\ .$$

Optimal Policy – In other words

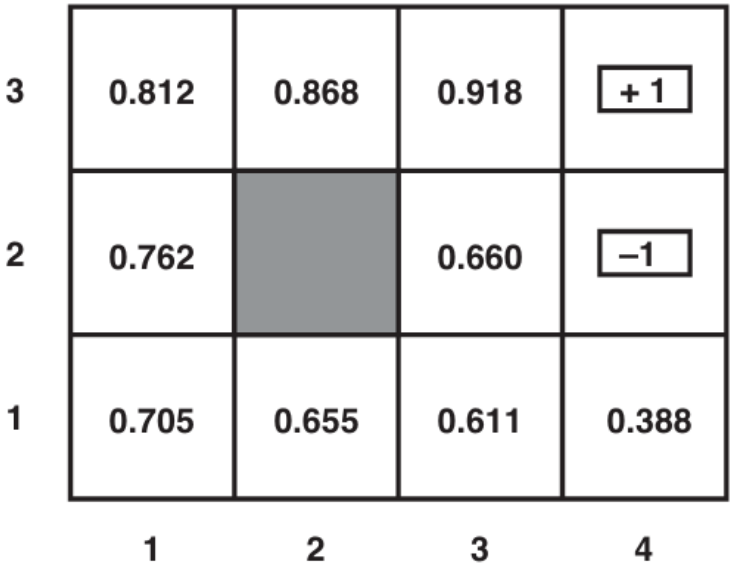
The optimal policy can be computed as:

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s' \mid s, a) U(s')$$

with A(s) being all possible actions from state s

Expected Utility – Bellman Equation

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a) U(s')$$



$$\begin{aligned} U(1,1) = -0.04 + \gamma \max[\; & 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1), && (Up) \\ & 0.9U(1,1) + 0.1U(1,2), && (Left) \\ & 0.9U(1,1) + 0.1U(2,1), && (Down) \\ & 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) \;]. && (Right) \end{aligned}$$

Value Iteration

Idea: Start with arbitrary utility values

Update to make them locally consistence with Bellman Equation

Everywhere locally consistent => global optimality

- set $U*_0(s) = 0$ for all s
- for i=0..h (or until convergence)
  - for all $s \in S$

$$U^*_{i+1} = \max_{a \in A(s)} \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma^i U^*_i(s')]$$

Value Iteration

Solving for $V^*(s)$

V and Q are defined recursively in terms of each other.

$$V^*(s) = R(s) + \gamma \max_a Q^*(s,a) \tag{3}$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) V^*(s'). \tag{4}$$

Combining equations 3 and 4, we get the Bellman equations:

$$V^*(s) = R(s) + \gamma \max_a \sum_{s'} P(s'|s,a) V^*(s'). \tag{5}$$

$V^*(s)$ are the unique solution to the Bellman equations.

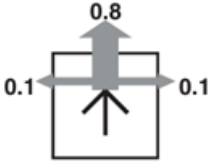
Value Iteration

Write down $V^*(s_{11})$

	1	2	3	4
1	0.705	0.655	0.611	0.388
2	0.762	X	0.660	-1
3	0.812	0.868	0.918	+1

Write down the Bellman equation for $V^*(s_{11})$.

$$V(s_{11}) = -0.04 + \gamma \max[0.8 * V(s_{12}) + 0.1 * V(s_{21}) + 0.1 * V(s_{11}), \quad \text{(right)}$$
$$0.9 * V(s_{11}) + 0.1 * V(s_{12}), \quad \text{(up)}$$
$$0.9 * V(s_{11}) + 0.1 * V(s_{21}), \quad \text{(left)}$$
$$0.8 * V(s_{21}) + 0.1 * V(s_{12}) + 0.1 * V(s_{11})]. \quad \text{(down)}$$



3	0.812	0.868	0.918	+1
2	0.762		0.660	−1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Value Iteration

CQ: Solve the Bellman equations efficiently

CQ: Can we solve the system of Bellman equations efficiently?

(A) Yes

(B) No

(C) I don't know

	1	2	3	4
1	0.705	0.655	0.611	0.388
2	0.762	X	0.660	-1
3	0.812	0.868	0.918	+1

The Bellman equation for $V^*(s_{11})$:

$$V^*(s_{11}) = -0.04 + \gamma \max[0.8V^*(s_{12}) + 0.1V^*(s_{21}) + 0.1V^*(s_{11}),$$
$$0.9V^*(s_{11}) + 0.1V^*(s_{12}),$$
$$0.9V^*(s_{11}) + 0.1V^*(s_{21}),$$
$$0.8V^*(s_{21}) + 0.1V^*(s_{12}) + 0.1V^*(s_{11})].$$

¨Value Iteration

Solving for $V^*(s)$ iteratively

The Bellman equations:

$$V^*(s) = R(s) + \gamma \max_a \sum_{s'} P(s'|s,a) V^*(s').$$

Let $V_i(s)$ be the values for the i-th iteration.

1. Start with arbitrary initial values for $V_0(s)$.
2. At the i-th iteration, compute $V_{i+1}(s)$ as follows.

$$V_{i+1}(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s'|s,a) V_i(s')$$

3. Terminate when $\max_s |V_i(s) - V_{i+1}(s)|$ is small enough.

If we apply the Bellman update infinitely often,
the V_i's are guaranteed to converge to the optimal values.

Policy Iteration

- Deriving the optimal policy does not require accurate estimates of the utility function V*(s)
- Policy iteration alternates between two steps
  - Policy Evaluation- Given a policy π_i, calculate $V^{\pi_i}(s)$, which is the utility of each state if policy π_i was executed
  - Policy improvement- Calculate a new policy π_{i+1} using V^{π_i}
- Terminates when there is no change in the policy

Policy Iteration v/s Bellman Equations

Policy Evaluation v.s. Bellman Equations

Policy evaluation:

$$V(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) V(s').$$

Bellman equations:

$$V(s) = R(s) + \gamma \max_{a} \sum_{s'} P(s'|s, a) V(s').$$

Write down both equations for $V(s_{11})$.
Assume that $\pi(s_{11}) = \text{down}$.

$$\mathrm{V}(S_{11}) = -0.04 + \gamma(0.8 * V(S_{21}) + 0.1 * V(S_{12}) + 0.1 * V(S_{11}))$$

Policy Iteration

Performing Policy Evaluation Exactly

Policy evaluation:

$$V(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) V(s').$$

Solve the system of linear equations exactly using standard linear algebra techniques.

For n states, this will take $O(n^3)$ time.

Policy Iteration

Performing Policy Evaluation Iteratively

Policy evaluation:

$$V(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) V(s').$$

Solve the system of linear equations approximately
by performing a number of simplified value iteration steps.

Policy Iteration

Policy Improvement

$$\pi_{i+1}(s) = argmax \sum P(s'|s,a)\, V^{\pi i}(si)$$

Policy Iteration

$$V_i(s) = R(s) + \gamma \sum P(s'|s, \pi_i(s)) V_i(s')$$

Thank You

Dr. Rajiv Misra, Professor
Dept. of Computer Science & Engg.
Indian Institute of Technology, Patna
rajivm@iitp.ac.in